

SISTEMA DE GESTIÓN DE INVENTARIOS BASADO EN POO.
PROYECTO FINAL POO.

Juan Sebastian Guerrero Vanegas - *Nota: 5.0*
Ervin Santiago Ardila Vanegas - *Nota: 5.0*
Juan Manuel Caviedes Sanchez - *Nota: 5.0*

Facultad de Ingeniería.
Universidad Nacional de Colombia - Sede Bogotá.

OBJETIVOS

Objetivo general:

Desarrollar un programa en Java que permita gestionar un inventario de productos mediante operaciones CRUD (crear, leer, actualizar y eliminar), utilizando los principios de la programación orientada a objetos mediante el uso de clases y objetos para organizar y manipular la información

Objetivos específicos:

- Implementar las operaciones CRUD para permitir el registro, consulta, modificación y eliminación de productos dentro del sistema de inventario.
- Organizar el código en estructuras claras de clases y métodos para facilitar el mantenimiento y la reutilización del programa.
- Diseñar clases, objetos y métodos para los diferentes objetos y procesos dentro del programa.

CONTEXTO Y JUSTIFICACIÓN DEL PROYECTO

Toda tienda o almacén requiere un control estricto de sus productos para registrar entradas y salidas de stock de manera eficiente. Actualmente, llevar este control de forma manual puede generar inconsistencias, pérdida de información y desabastecimiento. Este proyecto se justifica en la necesidad de automatizar la gestión del inventario mediante una herramienta de software basada en Java que garantice la integridad de los datos y facilite la toma de decisiones.

ENLACES DEL PROYECTO

Enlace al video:

https://drive.google.com/file/d/151tY4_H73QIk7D85a6JkQ0sA0o2n9LCS/view?usp=sharing

Enlace al repositorio de GitHub:

<https://github.com/jguerrerova-glitch/-SISTEMA-DE-GESTI-N-DE-INVENTARIOS-BASADO-EN-POO./blob/f1051bacd55a6978e951751a6cc1e975718bbb4b/README.md>

REQUISITOS FUNCIONALES

- El sistema debe permitir registrar nuevos productos en el inventario con sus respectivos atributos (ID, nombre, precio, cantidad).
- El sistema debe permitir consultar la información completa de los productos registrados.
- El sistema debe permitir actualizar la información de un producto, especialmente las entradas y salidas de stock.
- El sistema debe permitir eliminar un producto del registro.

REQUISITOS NO FUNCIONALES

- El sistema debe estar desarrollado en el lenguaje de programación Java.
- El diseño del código debe aplicar estrictamente los principios de la Programación Orientada a Objetos.
- El control de versiones del proyecto debe realizarse a través de un repositorio remoto en GitHub.

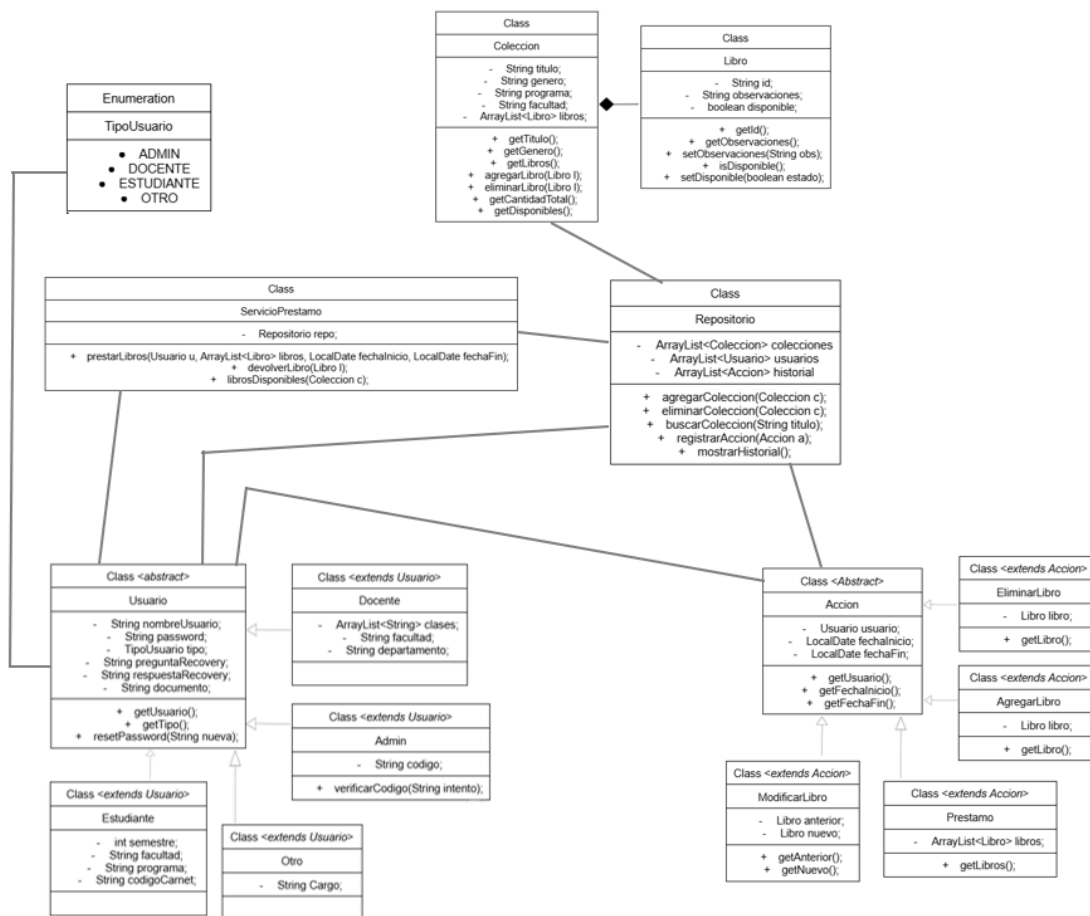
HISTORIAS DE USUARIO

HU-01	Registro detallado de productos	Como Administrador del almacén, deseo registrar un nuevo producto en el sistema especificando su ID, nombre, categoría, precio, proveedor y cantidad inicial, para ampliar el catálogo disponible y mantener el control del inventario físico.	1. El sistema debe validar que el ID ingresado sea único y no exista ya en el inventario. 2. El precio y la cantidad en stock inicial no pueden ser números negativos. 3. Si algún campo obligatorio (como el nombre o el ID) está vacío, el sistema debe arrojar un mensaje de error claro y no guardar el producto.
HU-02	Búsqueda y verificación de disponibilidad	Como Usuario del sistema (Administrador o Empleado), deseo buscar un producto utilizando su identificador (ID) o su nombre, para verificar rápidamente su precio y las unidades disponibles en bodega al momento de una venta o auditoría.	1. Si el producto existe, el sistema debe mostrar en pantalla todos sus detalles (ID, nombre, categoría, precio y stock actual). 2. Si el producto no se encuentra, el sistema debe mostrar el mensaje: "Error: El producto buscado

			no existe en el inventario".
HU-03	Registro de transacciones de inventario	Como Administrador del almacén, deseo registrar las entradas (abastecimiento) y salidas (ventas) de un producto específico, para asegurar que el stock real coincida con el stock del sistema en todo momento.	1. Al registrar una salida, el sistema debe verificar que haya suficiente stock. Si se intentan sacar más unidades de las que hay, se debe cancelar la operación y mostrar una alerta como "Stock insuficiente". 2. La cantidad a sumar o restar debe ser obligatoriamente un número mayor a cero.
HU-04	Depuración del catálogo	Como Administrador del almacén, deseo eliminar un producto del sistema que ya no se comercializará o que esté descontinuado por el proveedor, para mantener el catálogo limpio	1. Antes de eliminar el producto definitivamente, el sistema debe solicitar una confirmación al usuario. 2. Esta acción sólo debe estar permitida para perfiles con rol de Administrador.
HU-05	Reporte de bajo stock	Como Administrador del almacén, deseo generar un reporte automático de todos los productos cuyo stock sea inferior a 10 unidades, para saber qué artículos debe solicitar urgentemente a los proveedores.	1. El sistema debe recorrer toda la lista de productos y mostrar únicamente aquellos con cantidad menor o igual a un límite definido (ej. 10). 2. Si no hay productos con bajo stock,

			mostrar un mensaje de "El inventario se encuentra excelente".
--	--	--	---

DIAGRAMA UML



Para nuestro proyecto realizamos un diagrama UML de clases y relaciones que permitiera comprender y graficar las clases que requerimos para el funcionamiento del mismo junto con sus respectivos atributos y métodos enlistados por signos - y + respectivamente.

En este diagrama podemos ver que 2 de las clases principales son abstractas las cuales son la clase Usuario y Accion, ya que no pueden crearse objetos de estas superclases sino de sus clases hijas, ya que estas sirven más a modo de esqueleto para sus clases hijas.

Esto se ve mostrado con la etiqueta `<abstract>` y las flechas con las puntas blancas que apuntan a las superclases que apuntan a ellas, en el caso de Accion esta cuenta con un total de 4 clases hijas las cuales son EliminarLibro, AgregarLibro, ModificarLibro y Prestamo, las 3 primeras clases cumplen las funciones de Crear, Actualizar y Eliminar del CRUD y la última permite registrar los préstamos, estas 4 clases servirán para crear un Arraylist con un historial de las acciones realizadas, se puede evidenciar que heredan con la etiqueta `<extends>`.

Asimismo la clase Usuario también es abstracta y es el esqueleto para los 4 tipos de usuario, Admin, Docente, Estudiante y Otro, cada una con los atributos y métodos de Usuario pero con ciertas diferencias ligeras. Además, tenemos una particularidad que es el Enumeration llamado TipoUsuario, este tipo de dato almacena valores fijos, haciendo que el valor tipo en la clase Usuario solo pueda adoptar 1 de los 4 valores pre definidos y limitados en este.

Por otro lado, otra relación particular existe entre Libro y Coleccion, en este caso es una relacion de composicion que es una relacion fuerte que indica que Coleccion esta compuesto meramente por Libros y sin Libros no puede existir una Coleccion, esta representada por una linea con un rombo negro que apunta al que esta compuesto por el otro.

Asimismo, la última relación clave a notar es las líneas continuas que representan Asociación, estas se ven principalmente entre la clase Repositorio y gran parte de las clases o Usuario y gran parte de las clases demostrando así su relacion con la información que almacenan otras clases.